MULTIMODAL AI CLINICAL DECISION SUPPORT SYSTEM & TRIAGE ENGINE

MASTER OF COMPUTER APPLICATIONS (MCA)

Submitted By

Rupesh Kumar Singh

Roll No: 2024179051

Batch: MCA (self-support)

Under the Guidance of

Dr. Selvi Ravindran Assistant Professor

DEPARTMENT OF INFORMATION SCIENCE & TECHNOLOGY

ANNA UNIVERSITY

Chennai – 600025

January 2026

Table of Contents

1. **Chapter 1: Introduction**.....1
   1.1 Background.....2
   1.2 Problem Statement.....3
   1.3 Motivation.....4
   1.4 Objectives.....5
   1.5 Scope.....6
2. **Chapter 2: Literature Survey**.....7
   2.1 Existing Work.....8
   2.2 Observations.....9
   2.3 Gaps.....10
3. **Chapter 3: System Architecture**.....11
   3.1 React Frontend.....12
   3.2 FastAPI Backend.....13
   3.3 LangGraph Orchestrator.....14
   3.4 Vision CNNs.....15
   3.5 RAG.....16
4. **Chapter 4: Module Description**.....17
   4.1 Multimodal Vision Analysis.....18
   4.2 Medical RAG.....19
   4.3 LangGraph Reasoner.....20
   4.4 Dynamic UI.....21

Multimodal AI Clinical Decision Support System & Triage Engine

CHAPTER 1

INTRODUCTION:

1.1 BACKGROUND

The rapid growth of web-based healthcare services has created a strong demand for intelligent triage systems capable of handling large volumes of patient interactions efficiently. During peak healthcare demands such as pandemics and emergency surges, traditional hospital triage systems face severe workload pressure, long waiting times, and inconsistent prioritization of patients. Web-based medical triage services aim to address these challenges by automating preliminary symptom assessment, prioritizing patient urgency, and supporting clinicians with structured reports.

Recent advances in Artificial Intelligence (AI), Large Language Models (LLMs), and multimodal data processing have enabled the development of intelligent backend systems that can analyze patient inputs in the form of text, voice, images, and videos. These systems are increasingly being integrated with Electronic Medical Records (EMR) and clinical decision support tools to enhance triage accuracy and reduce documentation workload.

The base architecture proposed in “Architecture of a Web-based Medical Triage Service” demonstrates how scalable backend services can support real-time triage, severity classification, and data management. Building upon this foundation, modern AI-driven systems such as Dr. Console aim to incorporate reasoning intelligence, safety controls, and automated clinical report generation.

1.2 PROBLEM STATEMENT:

Despite the increasing availability and integration of web-based diagnostic and triage platforms in modern healthcare, several critical limitations persist that hinder their efficacy and widespread clinical adoption. Primarily, current systems exhibit a heavy dependence on rule-based logic or static classification models. This rigid architecture restricts their ability to

adapt to complex, nuanced patient presentations, often resulting in inaccurate or overly generalized triage advice. When patients present with multi-faceted or atypical symptoms, these systems lack the deep reasoning capabilities required for accurate clinical interpretation, failing to mimic the deductive processes of a human medical professional.

Furthermore, a significant challenge in contemporary AI-driven healthcare tools is the absence of reliable medical knowledge grounding. Many existing large language models operate as "black boxes," generating responses based on generalized training data rather than validated clinical guidelines. This lack of grounding introduces a severe risk of unsafe or hallucinated AI outputs, wherein the system might confidently provide incorrect, irrelevant, or potentially harmful medical advice. In a high-stakes clinical environment, such unreliability is unacceptable and serves as a major barrier to trust for both patients and healthcare providers.

Additionally, existing triage systems often fail to alleviate the administrative burden on healthcare workers. The process of manually transcribing patient interactions into standardized clinical documentation (such as SOAP notes) remains a time-consuming bottleneck, diverting valuable time away from direct patient care

Collectively, these limitations—static reasoning, propensity for hallucinations, lack of verifiable grounding, and administrative inefficiency—drastically reduce the reliability of automated triage systems. Consequently, there is an urgent need for an intelligent backend architecture that transcends basic symptom matching. The proposed Multimodal AI Clinical Decision Support System & Triage Engine seeks to address these gaps by integrating advanced reasoning, retrieval-augmented grounding, stringent safety enforcement, and automated report generation into a single, unified, and highly reliable triage pipeline.

1.3 MOTIVATION:

The motivation for this project arises from the growing adoption of AI-powered clinical assistants and the increasing workload faced by healthcare professionals. By integrating LLM-based reasoning, Medical Retrieval-Augmented Generation (RAG), and multimodal analysis within a coordinated backend architecture, triage systems can become more accurate, explainable, and clinically useful.

The Dr. Console backend is motivated by the need to move beyond basic chatbot-style symptom checkers toward structured, safety-aware clinical decision support engines capable of producing triage summaries and SOAP reports.

1.4 OBJECTIVES :

The primary objectives of this project are:

1) To design a scalable backend architecture for web-based medical triage
2) To integrate AI reasoning engines for symptom interpretation
3) To implement Medical RAG for evidence-grounded responses
4) To automate SOAP(Subjective Objective Assessment Plan) note and triage report generation.
5) To enforce safety and alignment mechanisms in AI outputs

1.5 SCOPE OF THE PROJECT:

The primary scope of project is concentrated on the development and integration of advanced backend intelligence and orchestration components. The overarching goal is to create a robust, secure, and highly capable digital triage pipeline that mimics the early stages of a clinical consultation. Specifically, the project encompasses the following core functional areas:

Coordinator-Based Routing of Triage Tasks: Implementing an intelligent orchestration layer (utilizing a state-machine architecture like LangGraph) to seamlessly manage the flow of the consultation. This coordinator dynamically assesses the patient's inputs in real-time, determines the necessary subsequent steps (e.g., requesting more information, invoking specialized medical tools, or triggering emergency protocols), and maintains the conversational state and medical memory throughout the session.

Multimodal Input Handling: Expanding the system's analytical capabilities beyond standard text-based symptom descriptions. The scope includes the integration and routing of medical imagery (specifically focusing on distinct anatomical regions such as the skin, eye, ear, and tongue). The system will ingest these multimodal inputs and route them to designated, fine-tuned deep learning classifiers for localized diagnostic pre-screening.

Clinical Reasoning Pipelines: Developing a sophisticated analytical core powered by Large Language Models (such as Gemini). This pipeline is responsible for synthesizing diverse data streams—including the patient's self-reported symptoms, output from the multimodal vision models, and historical medical profile data—into a cohesive, clinically accurate preliminary assessment.

Knowledge Retrieval Systems: Integrating a Retrieval-Augmented Generation (RAG) framework to ensure the AI's reasoning is firmly anchored in verified medical literature. The scope involves building a vector database populated with trusted resources (e.g., the MedQuAD dataset) and semantic search mechanisms to actively fetch and inject relevant clinical guidelines into the AI's decision-making process, thereby mitigating hallucinations.

Automated Report Generation Modules: Engineering a secure module dedicated to synthesizing the entire triage interaction into standardized clinical documentation. This includes the automatic generation of structured SOAP (Subjective, Objective, Assessment, Plan) notes, readying the patient's data for rapid handoff to human medical professionals.

CHAPTER 2

LITERATURE SURVEY:

2.1 EXISTING WORK:

WEB-BASED MEDICAL TRIAGE ARCHITECTURE

Harby et al. proposed the Architecture of a Web-based Medical Triage Service, introducing a scalable Triage-Bot system designed to assist emergency healthcare services. The architecture supports multimodal user input, secure authentication, disease prediction, severity classification using the CTAS scale, and backend data analytics pipelines.

The system integrates MongoDB for flexible data storage and employs a service-oriented backend for handling triage requests. While the architecture demonstrates scalability and data integration capabilities, it primarily relies on traditional machine learning classifiers and predefined workflows. The system lacks deep reasoning modules and structured generative report generation mechanisms.

2.2 LLM-BASED CLINICAL REASONING SYSTEMS

Singhal et al. demonstrated that large language models such as Med-PaLM can encode substantial clinical knowledge and achieve strong performance on medical questionanswering benchmarks including MedQA and MultiMedQA. Their findings highlight the potential of LLMs to support backend reasoning tasks such as symptom interpretation, diagnosis suggestion, and clinical summarization.

However, the study also reports risks related to hallucinations and safety inconsistencies, emphasizing the need for controlled deployment frameworks. This motivates the integration of LLMs within orchestrated backend pipelines rather than using them as standalone decision-makers.

2.3 CAUSAL RULE EXTRACTION FOR MEDICAL DECISION SUPPORT

Rahman et al. proposed a framework for extracting causal relational rules from medical question-answering datasets using LLM prompting techniques. The approach enables automated generation of symptom-to-treatment and symptom-to-diagnosis relationships without manual annotation.

Such causal modeling is valuable for backend triage engines as it enables structured reasoning paths, explainable decisions, and improved follow-up question generation. Incorporating causal relationships strengthens the interpretability and reliability of AIassisted triage workflows.

2.4 AI SAFETY AND ALIGNMENT IN MEDICAL APPLICATIONS:

Bai et al. introduced the Constitutional AI framework, which applies rule-based principles to guide safe AI behavior without heavy reliance on human feedback datasets. The framework emphasizes transparency, harmlessness, and controlled response generation.

Medical triage systems operate in safety-critical environments where incorrect outputs may cause patient harm. Applying constitutional-style safety constraints in backend reasoning pipelines enables controlled refusal generation, escalation handling, and safer response filtering.

2.5 CHAIN-OF-THOUGHT REASONING FOR MULTI-STEP CLINICAL TASKS:

Che et al. proposed a dynamic chain-of-thought prompting framework that improves multistep reasoning accuracy by integrating external computation modules. Their approach demonstrates improved reasoning performance across complex tasks.

For medical triage systems, chain-of-thought reasoning improves structured symptom analysis, diagnostic decomposition, and report generation reliability. This technique supports backend modules responsible for SOAP note synthesis and triage justification generation.

2.6 AI-ASSISTED MEDICAL REPORT GENERATION:

Ramoni et al. analyzed the use of AI systems for medical documentation and scientific writing. Their study highlights productivity benefits while emphasizing ethical risks associated with hallucinated content.

Automated report generation modules in triage systems must therefore integrate template constraints, medical knowledge grounding, and validation layers to ensure clinical reliability.

2.7 OBSERVATIONS FROM EXISTING WORK

From the surveyed literature, it is observed that:

· Web-based triage architectures provide scalability but lack deep reasoning

· LLMs demonstrate strong clinical knowledge representation

· Causal reasoning improves explainability

· Safety alignment mechanisms are essential for deployment

· Automated documentation reduces clinician workload

2.8 LIMITATIONS AND RESEARCH GAP:

Although significant progress has been made, existing systems exhibit important limitations:

· Fragmented AI pipelines without centralized orchestration

· Limited integration of reasoning and retrieval systems

· Lack of structured backend-driven SOAP report generation

· Insufficient safety enforcement layers

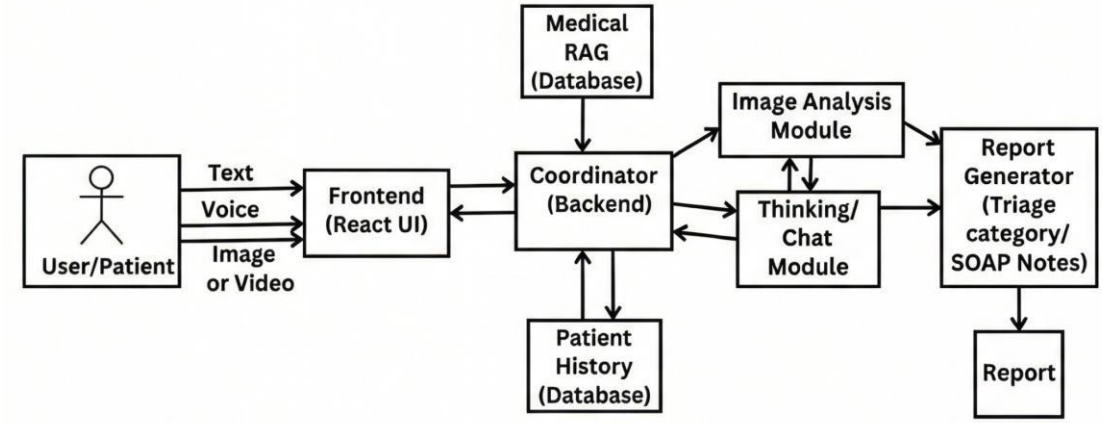
Research Gap: There is a need for a unified backend architecture that combines triage orchestration, LLM reasoning, medical knowledge grounding, safety enforcement, and automated clinical report generation into a single coordinated pipeline.

2.9 SUMMARY OF LITERATURE SURVEY:

In summary, existing research demonstrates the effectiveness of web-based triage systems and AI-powered clinical reasoning. However, most architectures focus on classification and data management rather than coordinated reasoning and report automation. The base paper provides a scalable architectural foundation, while recent LLM and safety research enables intelligent backend extensions. The proposed Dr. Console backend architecture addresses these gaps by integrating orchestration, reasoning, retrieval, safety control, and structured report generation.

CHAPTER 3

SYSTEM ARCHITECTURE



.

The architecture has four main layers:

1. First, the **React frontend** provides a user interface in the browser[9]. React is a declarative JavaScript library for building interactive UIs[9]. The frontend manages forms for uploading medical images (eye, ear, tongue, skin) and recording patient audio or text notes. It uses local storage to **persist session state** (e.g. partially entered data) so the user can navigate away and return without losing context[10]. The UI also displays retrieved medical information and chatbot responses. Components update dynamically as data becomes available. Upon submitting inputs, the frontend makes REST API calls to the backend.

2. The second layer is the **FastAPI backend** in Python. FastAPI is a modern, high-performance framework for building RESTful APIs[11]. The backend defines endpoints to receive image files, audio, and text from the frontend. Each request

is routed to the orchestrator for processing. FastAPI's asynchronous request handling ensures low latency for I/O-bound tasks such as loading images or querying databases. Input data (images, audio) are sent to the orchestrator or vision modules as appropriate. The backend also serves the React app and provides interactive API documentation via OpenAPI (Swagger) for developers.

3. The core logic resides in the **LangGraph orchestrator** layer[8]. LangGraph manages a directed graph of agent nodes for decision-making. The orchestrator maintains a shared **state** object (e.g. user inputs, current analysis results) and defines **nodes** that perform computations, such as invoking the Gemini LLM or running a vision model. **Edges** in the graph define conditional flows: for example, if the vision models detect an emergency pattern, the orchestrator can branch to an "alert" node that escalates the issue. We implemented a ReAct-style agent: the graph alternates between "thinking" steps (LLM-generated plans or queries) and "acting" steps (tool calls). For example, one node may call the eye-model on an uploaded ocular image, then update the state with findings; the LLM node may then query the RAG knowledge base for related treatment info. LangGraph's structure allows complex, stateful reasoning while remaining modular[8].

4. Under this orchestration are the **specialized vision models**. We use four MobileNetV2-based CNNs, each trained to recognize patterns in one modality: an ocular model (detects pupil irregularities, hemorrhages, jaundice), an auricular model (ear canal abnormalities), a lingual model (tongue texture/discoloration), and a dermal model (skin lesions)[3][2][4][5]. MobileNetV2 architectures are chosen for their efficiency on mobile images[2][3]. Each model outputs a classification or feature vector indicating normal vs. concerning findings. The orchestrator receives these outputs to update the state. For instance, if the tongue model flags a disease, the state adds that finding as a symptom for the LLM to consider. These CNNs run locally on the backend server (or could

   be served as microservices), taking image arrays as input and returning numerical scores.

5. Finally, the system includes a **Retrieval-Augmented Generation (RAG)** knowledge base. We store the MedQuAD medical QA content as documents in

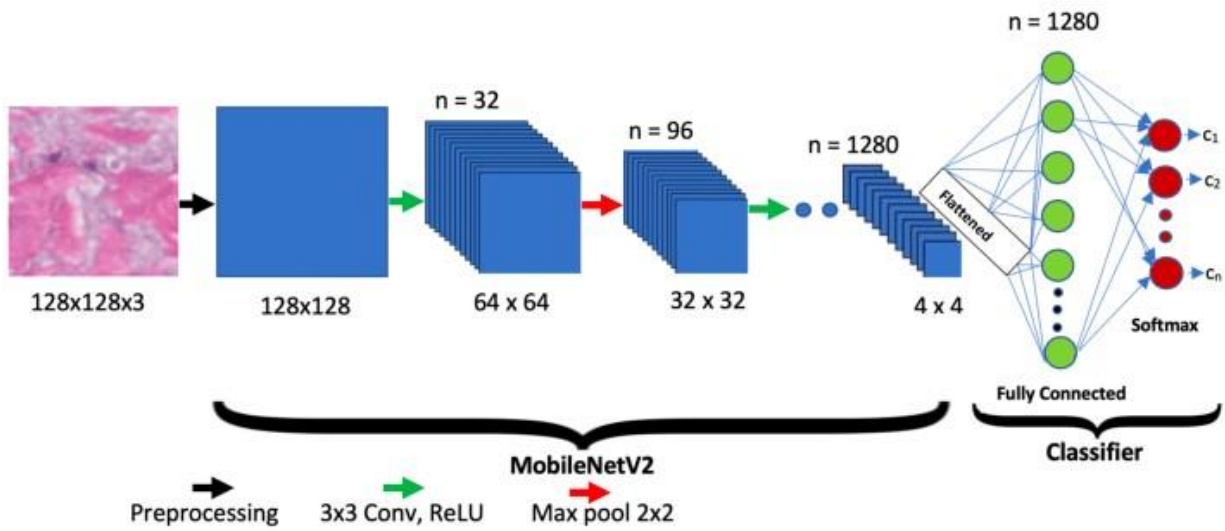
a PostgreSQL-backed vector store (Supabase with pgvector)[12]. During preprocessing, each Q&A pair from MedQuAD is embedded using the sentence-transformer *all-MiniLM-L6-v2* model, which produces 384-dim vectors efficiently[13]. These embeddings are indexed in Supabase. When the user or LLM poses a medical question (e.g., "What causes tongue discoloration?"), the orchestrator embeds the query (using the same all-MiniLM-L6-v2) and issues a vector similarity search. The database returns the top matching Q&A pairs as context. This retrieved context is then fed into the Gemini LLM to produce a final answer that cites specific medical sources. In this way, the LLM's generation is grounded in factual data (following RAG methodology[6]).

In summary, Figure 1 illustrates how data flows: the React UI sends inputs to FastAPI, which invokes LangGraph. The orchestrator calls the vision CNNs and the RAG service, then calls Gemini to reason over both observed data and retrieved knowledge. The combined output (diagnosis, suggestions, or alerts) is sent back to the frontend to display to the user. Each component leverages proven technologies: React[9] for UI, FastAPI[11] for API, LangGraph[8] for LLM orchestration, MobileNetV2 for vision[2], and Supabase/pgvector for RAG storage[12].

CHAPTER 4

MODULE DESCRIPTION

4.1 Module 1: Multimodal Vision Analysis



This module implements the **pipeline for image-based diagnosis**. It handles data acquisition, preprocessing, and CNN inference for each modality:

- **Datasets:** We leverage labeled medical image datasets for training the four MobileNetV2 models. For example, the tongue model is trained on a smartphone-captured tongue disease dataset[3]. The eye model uses public retinal image datasets (e.g. [ROCKED], [Kaggle DR]) where diabetic retinopathy and other pathologies are annotated[2]. The skin model is fine-tuned on dermoscopic lesion images (HAM10000, PH2, etc.), as done in recent studies[4]. For the ear, we use curated otoscopic image sets with labeled conditions (normal, otitis media, etc.)[5]. In each case, thousands of examples are collected to train the CNN.

- **Preprocessing:** Incoming images (RGB photographs) are resized and normalized to the input size of the MobileNetV2 networks (e.g. 224×224 pixels). We apply standard augmentation (random flips, color jitter) during training to improve robustness. Images are converted to arrays and batched

before inference. Each image is passed to the appropriate model based on the input type (the frontend indicates whether an image is of the eye, ear, tongue, or skin).

- **Model Inference:** Each modality has its own MobileNetV2 variant (with final dense layers adapted for binary/multilabel classification). For example, the tongue model first classifies "healthy vs. diseased tongue" and then, if diseased, applies a second DenseNet-121 network to predict specific conditions[3]. Similarly, the skin model distinguishes malignant vs. benign lesions with >90% accuracy[4]. The models output confidence scores and predicted classes (e.g. "retina: diabetic retinopathy").

- **Execution Pipeline:** After classification, the module extracts the relevant outputs and stores them in the global state for LangGraph. For instance, if the ear model predicts "otitis media with effusion" with high confidence, this finding is added to the patient's symptom list. All vision inferences occur before the LLM step; they serve as "tools" for the agent. In case multiple images are provided, the module processes them sequentially. The pipeline is as follows:

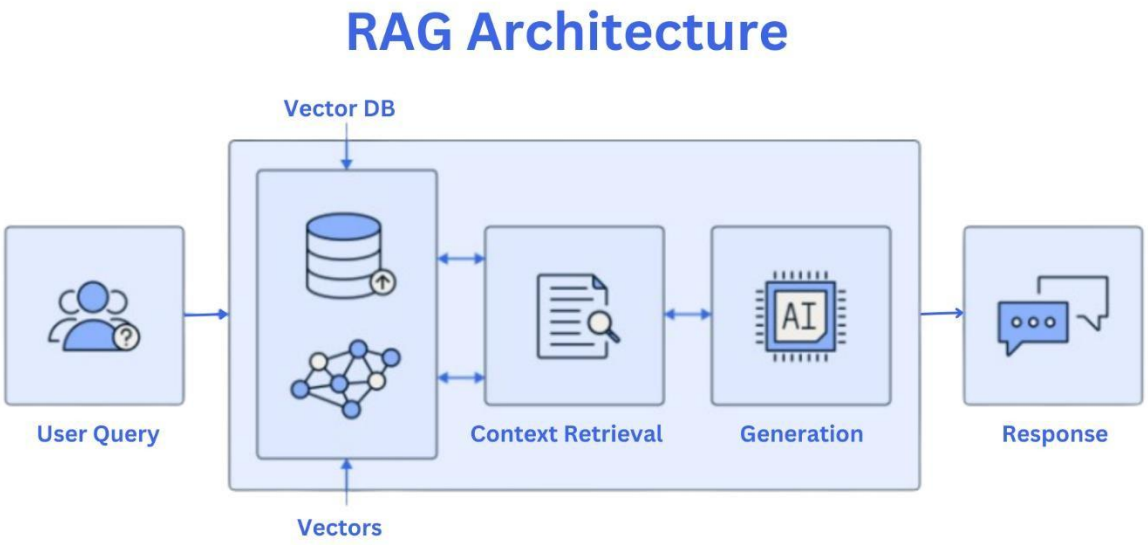
- Receive image and modality label from frontend.
- Preprocess image (resize, normalize).
- Run through the corresponding MobileNetV2 model.
- Output class label(s) and confidence.
- Update orchestrator state with findings.

Intermediate Output:


```
{
  "detected_specialist": "eye",
  "raw_prediction_tensor": [[0.02, 0.05, 0.91, 0.02]],   "classification": {
    "condition": "Conjunctivitis (Pink Eye)",
    "confidence": 91.0,
    "is_high_risk": false,
    "warning": null
  }
}
```


This visual analysis module is data-driven and performs the first level of diagnosis. Research shows such CNN-based analysis is effective: e.g., a lightweight MobileNetV2 model achieved 90.8% accuracy on retinal disease detection[2], and 98.6% accuracy on skin lesion classification[4]. The tongue model in [3] achieved ~93% accuracy on disease vs. healthy, demonstrating the viability of smartphone image analysis. These results give confidence that the module can reliably flag significant clinical signs from images, which are then interpreted by subsequent modules.

4.2 Module 2: Medical RAG



This module handles **retrieval-augmented question answering**. It ingests textual queries (either from the LLM agent or the user) and returns relevant medical information:

- **Knowledge Embedding:** We use the MedQuAD dataset[7] as our Q&A source. All MedQuAD question-answer pairs are embedded offline using the all-MiniLM-L6-v2 sentence-transformer model[13]. These 384dimensional embeddings are stored in a Supabase (Postgres+pgvector) database[12]. Supabase provides an open-source toolkit for managing vector embeddings at scale[12].
- **Query Embedding and Retrieval:** When a query arrives, we embed the query text using the same all-MiniLM-L6-v2 model. We then perform a nearest-neighbor search in the Supabase vector table, retrieving the top-k most similar Q&A entries. This yields factually relevant passages (answers) from MedQuAD. The retrieval uses cosine similarity and returns both the question and answer fields of each matched entry.

- **Answer Generation:** The retrieved passages are combined into context and provided to the LLM (Gemini) as evidence. For example, if the patient data include “yellow skin” and the question is “What causes jaundice?”, the system might retrieve MedQuAD answers about bilirubin and hepatitis. The LLM is prompted to generate a coherent response incorporating these facts. This ensures that the answer is grounded in the medical sources[6][7].

- **RAG Logic:** The module also contains logic for deciding when to query the knowledge base. Typically, queries arise from the LangGraph agent (e.g. “What treatment is appropriate for diabetic retinopathy?”) or from an emergency rule trigger. In all cases, the workflow is:

- Receive a query string from the agent.
- Embed the query with all-MiniLM-L6-v2.
- Retrieve top-N similar MedQuAD entries via Supabase.
- Concatenate retrieved answers (and questions if needed) into the prompt for the LLM.
- Return the LLM’s answer to the orchestrator.

Intermediate Output:


```
[
 {
  "topic": "Glaucoma",
  "content": "If left untreated, glaucoma can cause progressive vision loss and eventual blindness. The damage to the optic nerve is irreversible...", "similarity_score": 0.84
 },
 {
  "topic": "Intraocular Pressure",
```

```
  "content": "High eye pressure is the main risk factor for optic nerve damage resulting from glaucoma...",
  "similarity_score": 0.76
 }
]
```

By using RAG, the system can provide **source-backed medical explanations** rather than hallucinated answers. The MedQuAD content, covering 37 question types[7], includes diseases, symptoms, treatments, etc., making it well-suited for patient queries. In practice, our RAG pipeline helps the LLM cite specifics (e.g. "According to NIH sources, …" as in MedQuAD). This module thus bridges raw patient data and domain knowledge, enabling accurate and up-to-date medical advice.

4.3 Module 3: LangGraph Thinking Orchestrator

The **LangGraph orchestrator** manages the decision-making flow and stateful logic of the agent. It consists of a state object and interconnected processing nodes:

- **State Logic:** The state includes user history (messages, inputs), vision analysis results, and flags for emergency conditions. For example, the state might contain entries like `{"eye_findings": "normal", "skin_findings": "lesion", "steps": 0}`. We define the state as a Pydantic `TypedDict` so that LangGraph can automatically update it when nodes emit new values. Shared state allows information to persist across steps[14].

- **Node Graph:** Each node in the graph corresponds to a computation or decision. Key nodes include:

- **Vision Nodes:** After image inputs, these nodes run the MobileNet models (as described in Module 1) and update the state with labels/scores.
- **Query Node:** Constructs a prompt from user intent (or detected symptoms) and calls the Gemini LLM with relevant data.
- **RAG Tool Node:** Given a text query in state, calls the RAG module to fetch context, then may trigger the LLM to incorporate those facts.

- **Emergency Check Node:** Evaluates vision and audio results against thresholds (e.g. if eye hemorrhage or extreme vital signs). If detected, it sets an `emergency_alert` flag.
- **Response Node:** Formats the final answer (text, images, or alerts) to send back to the user.

- **Edges and Control Flow:** Edges define which node runs next based on the current state. For instance, if no emergency is flagged, the next node might be "Generate Report" (LLM); if an emergency is detected, an alternative edge leads to "Raise Alarm". In ReAct fashion[8], nodes alternate between "thought" (LLM reasoning) and "action" (running a tool or returning results). This allows conditional logic. For example, after initial analysis, the LLM node might ask itself a sub-question ("Should I consult the RAG database?") and direct control to the RAG node accordingly.

- **Implementation:** We implemented the orchestrator using the LangGraph library. States are defined as typed dicts, nodes as async functions, and edges via simple conditionals on the state. This structured approach makes the system's reasoning transparent and debuggable. Each interaction ends when the agent reaches a terminal node (e.g. final answer or alarm).

Overall, the LangGraph orchestrator ensures the system can **think and act** in a loop, using tools as needed. It embodies the "chain-of-thought" with tool usage, providing a flexible yet controlled workflow for the healthcare AI agent[8].

4.4 Module 4: Dynamic Frontend UI & Persistence

The frontend module implements a responsive web interface with data persistence:

- **Dynamic Rendering:** React components render based on state. For example, after the backend returns analysis results, a "Results" component displays text diagnostics and annotated images. We use React's component state and hooks to trigger re-renders when new data arrives. If the user submits multiple images, the UI dynamically updates to show each result as it completes. This creates an interactive feel. The UI also includes a chat-like

panel where the LLM's answers (pulled from Module 3) appear sequentially, with citations if needed. No page reloads are required.

- **Session Persistence:** To prevent data loss, the application uses the browser's localStorage/sessionStorage APIs. Key user inputs (uploaded images, entered text notes, conversation history) are saved to localStorage so that they survive page refresh or navigation. For instance, we implemented a custom React hook (`useLocalStorage`) that wraps `useState` but automatically syncs to `window.localStorage`[10]. On each change to important state (e.g. user form fields), the data is stringified and stored. On app load, the React app checks localStorage and restores any saved state. This ensures that a user can resume their session without re-uploading. This pattern of combining component state with localStorage has been shown to provide "persistence by default" in web apps[10].

- **RAG Rendering:** When the RAG Module produces a text answer, the frontend renders it in a formatted block. If the answer includes lists or references, we display them using markdown or HTML elements. We also highlight key terms (e.g. diseases) for clarity. Any images or charts generated by the backend (e.g. heatmaps) can be embedded. The UI thus acts as a live report viewer.

In summary, Module 4 ensures a seamless user experience. The combination of React's dynamic rendering and browser storage means users interact fluidly, and their data/state persist. This complements the AI backend modules by providing a reliable interface for input/output.

REFERENCES:

[1] A. A. Harby et al., “Architecture of a Web-based Medical Triage Service,” IEEE COMPSAC, 2024.

[2] K. Singhal et al., “Large Language Models Encode Clinical Knowledge,” Nature, 2023.

[3] M. S. Rahman et al., “Extracting Causal Relational Rules for Medical QuestionAnswering Tasks using Large Language Models,” ACM CHASE, 2025.

[4] X. Che et al., “Chain-of-Thought Driven Dynamic Prompting and Computation Method,” Applied Soft Computing, 2025.

[5] D. Ramoni et al., “Artificial Intelligence in Scientific Medical Writing,” European Journal of Internal Medicine, 2024.